

MarginStream

A cross-margin derivatives venue with separated margin and matching authorities

SC6118 Scalable Systems Architecture for Fintech — Capstone, Group 1

LIU KEFAN ZHOU CONGXIANG WU YOUJHEN ZHANG HAOXI

Executive summary

MarginStream is a futures venue, with 40 underlyings and up to $20\times$ leverage, where one deposit backs all of a client's positions, so a client long one contract and short a correlated one pays margin only on the net risk. That capital efficiency is the product, but it is hard to deliver: "margin must not exceed equity" applies to the whole account, while order books are split by symbol and run in parallel. Each order must be accepted or refused in microseconds, yet no single book sees the whole account, and checking each book separately is unsafe — in our simulator, an account with 2,000 of collateral reached a requirement of 201,000 (E2).

Our solution moves the account-level calculation off the order path. A margin allocator gives each gateway a budget per account every 50–200 ms, and the gateway checks orders against it locally. The budgets are sized so that, even if all are used in full and the market makes the worst move our model covers, the account can still meet its margin, while a single ordering point records every accepted order and can revoke any gateway's authority at once. When things fail, the system stops rather than guesses. If the allocator is unreachable, gateways trade within their current budgets until they expire, then stop. If the market moves beyond the model, the venue takes over the account's positions in atomic basket transfers, and any shortfall falls to the insurance fund and, as a last resort, auto-deleveraging. After a crash, state is rebuilt by replaying one ordered log; 3,642 injected crashes all recovered exactly (E4).

This safety has two costs. First, budgets keep the account solvent after the worst covered move without liquidation, which roughly doubles the margin held, much as a venue sets initial margin above maintenance margin; a central checker with the same guarantee would pay this too. Second, a hedge split across two gateways cannot offset within either budget, which is the true cost of distribution. In our tightest test (E1), budgets were 99% used and the requirement reached 49% of equity, with no breach. We built and tested a deterministic simulator of admission, recovery and liquidation, with 11 test suites and 9 experiments including a flash crash (E9); replication, the production ledger, the mark-price pipeline and the insurance-fund waterfall are designed but not built, as §5.7 sets out.

1. Business context and requirements

1.1 The venue

MarginStream is a derivatives venue offering linear perpetual and dated futures on 40 underlyings, 120 contracts, to APAC retail and institutional clients, all margined against a single account balance rather than contract by contract. Leverage is capped at $20\times$; parameters below assume a median active account at $8\times$. The commercial reason for a unified account is capital efficiency — a client long one contract and short a correlated one should not fund both legs separately — so any design that quietly removes the offset removes the reason the venue exists, and the cost of every conservative step below is stated as a number.

Scale: 10^6 registered accounts, 10^5 with open positions, 10^4 whose positions or marks change between two issuances. A median account holds 5 contracts; the tail holds 50.

1.2 The requirement that does not decompose

The running case places pre-trade risk before the sequencer because the check is per-account and per-asset: freezing funds for one order says nothing about any other book, so it runs per connection and scales horizontally (Part 5 §1). Matching is sharded by symbol, single writer per shard (Part 2 §3).

A unified cross-margin account removes the first property and leaves the second. A fill on one contract changes the requirement of positions in other contracts held by the same account, sitting on shards being written concurrently. The invariant

the account's margin requirement must not exceed its equity

is global over the account and non-additive over contracts, yet it must be enforced before the order reaches the book, inside the same latency budget.

Locking the account, giving each holder a fixed sub-limit with no offset, and admitting optimistically are the three obvious resolutions; ADR-1 records why each fails.

§2 keeps matching exactly as the running case has it and moves the difficulty into a second authority: a margin allocator that runs off the order path and hands **each ingress gateway** a locally checkable share of the account's capacity. Matching shards hold no lease and make no margin decision.

Where this sits relative to the nearest candidate system. MarginStream is candidate 3 — a real-time risk and liquidation engine for a margin venue — with a harder core. Candidate 3 assumes the account-level check has already been serialised somewhere; here it cannot be, because the books it depends on are sharded by symbol and written concurrently. Candidate 3's obligations still apply and are met here: the mark-price pipeline (§2.5), the liquidation waterfall from unwind through the venue book and insurance fund to ADL (§5.4), and a flash crash run as a test vector (E9, §5.4).

1.3 Scope

Designed here: the venue's requirements, the margin authority and admission path, the consistency map, the derivatives accounting model, the degradation ladder, recovery, the liquidation waterfall, the threat model, operations and the target deployment. Out of scope: order routing, market making, fiat rails, wallet infrastructure.

Four things are taken from the running case and cited rather than re-derived. What matters is where each sits and what the margin authority does to it:

Cited from the running case	Where it sits here (Figure 1)	How the margin authority interacts with it
Matching engine, single writer per symbol (Part 2 §3)	behind the ordering point, eight shards	none on the order path: a shard holds no lease and makes no margin decision. Liquidation does not use it; baskets are internal transfers (§5.4)
Replicated log (Part 4 §2–3)	the ordering point's log	the allocator writes lease inputs to it and reads occupancy from it; gateways derive their budgets from it; fences, seals and barriers are records on it
Determinism (Part 2 §3)	every component's state is a fold of the log	budgets are derived deterministically from logged inputs, so they are not logged themselves (ADR-4); all arithmetic is integer
Double-entry ledger (Part 3 §1)	downstream of fills and baskets	a lease never appears in it; USER_MARGIN_HOLD replaces per-order USER_HOLD; funding, the insurance fund and ADL are new postings (§4.4)

1.4 Non-functional requirements

#	Requirement	Target	Consequence for the design
1	Order throughput	100k/s sustained, 1M/s burst	Admission is a local array operation; no allocator call on the order path
2	Admission-path latency	p50 < 20 µs, p99 < 200 µs, above matching	Per-account per-gateway scenario vector kept resident. Argued, not measured: E3 supports the scaling, not the target
3	Matching latency	p50 < 100 µs, p99 < 1 ms in-engine	Unchanged; single writer per symbol
4	Margin correctness	Requirement never exceeds equity after any move the scenario grid covers	The closure of §2.2, with the factor of two shown tight
5	Tightening latency	Binds within the lease term under partition, immediately when the ordering point is reachable	Fence at the ordering point; §1.5
6	Allocator cadence	Issuance every 50–200 ms	§1.5, §1.6
7	Allocator throughput	≈ 3 × 10 ⁷ scenario operations per issuance	Sharded by account; grid evaluated as a vector
8	Availability	99.99% for order entry; failover < 3 s	Replicated log, as in the running case
9	Degradation	The venue can bound its own loss in every state above HALT. Client-initiated risk reduction is not preserved under partition	Venue-initiated liquidation (§5.4); §3.3 states what is given up

#	Requirement	Target	Consequence for the design
10	Auditability	Target: every admission <i>and refusal</i> replayable with the figures it compared. Today: admissions only. A refusal never reaches the ordering point, and §2.6's gateway refusal journal is not built	
11	Solvency	Target: Σ user liabilities $\leq \Sigma$ venue assets, continuously checkable. Today: three facts about one account (§4.3). The ledger is unimplemented and the fund's size is argued from one stress run (§5.4), so the venue-level claim is an argument	

Rows 1, 3 and 8 are inherited; 2, 6 and 7 derived in §1.5–1.7; 4, 5 and 9 established in §2, §3 and §5. Rows 10 and 11 are targets, marked as such; §9.4 lists everything designed and not built.

1.5 What sets the lease term

The term is the main operational trade-off, and it is bounded from both sides. **Recompute cost is the floor:** $\approx 3 \times 10^7$ operations per issuance (§1.6) is ordinary at a 100 ms cadence and not at 1 ms. **Tightening latency is the ceiling:** a gateway nobody can reach keeps spending its budget until the term ends.

The lease term is the worst-case delay before a credit cut binds on a gateway the allocator cannot reach.

Raising or lowering a limit on a reachable gateway takes effect at the next issuance; a fence at the ordering point (§2.4) binds immediately on any gateway, reachable or not, and is kept for the heavy cases. 50–200 ms is a design target against an assumed tightening SLO, not a regulatory figure.

1.6 What the allocator has to compute, and how often

Each account needs about 3×10^3 operations per issuance: one pass over the scenario set, one check of §2.2's condition and a bisection for the budget size. With 10^4 accounts changing per issuance, that is $\approx 3 \times 10^7$ **operations per issuance**, or $\approx 3 \times 10^8$ per second at a 100 ms cadence, which sixteen allocator shards (§2.6) carry with headroom. No invariant spans two accounts, so sharding by account is trivial, and unchanged accounts keep their budgets.

1.7 What the admission path may do per order

Admission never recomputes the requirement from positions. Each gateway keeps running worst-case totals per account, about 128 bytes each, or ≈ 64 MB for 5×10^5 account and gateway pairs, and admitting is one pass over the scenario set and three integer comparisons. E3 confirms the scaling: ten times more live orders changed the incremental check by 1.1% while a full scan grew 6.7×, and widening the set from 7 to 16 scenarios cost 34%. These are CPython ratios, so NFR row 2's absolute target remains argued.

2. Architecture

2.1 Topology

Figure 1 gives the component view. Orders reach the venue through N ingress admission gateways, pass through a single **ordering point**, and are forwarded to a matching core partitioned by symbol. Matching keeps a single-writer total order per symbol and no order crosses a shard; that is the running case's arrangement and this design does not touch it.

What is added is a second authority. The **margin allocator** runs off the order path, computes each account's capacity, and hands each gateway a locally checkable share of it. A gateway decides in constant time from what it already holds; it calls no allocator and reads no other gateway's state.

	Partition key	Why
Matching	Symbol	Price-time priority is a total order per book, so a book must have one writer
Allocator	Account	Margin is an account-level quantity, and no invariant spans two accounts

The partitionings are orthogonal, which is the structural point: an account's positions spread across matching shards, a shard holds many accounts, and neither needs the other's partition to be correct. The single-writer core is plural — one writer per symbol for the book, one per account for the capacity.

2.2 The idea in one page

Why the requirement cannot be checked one book at a time. A client long BTC-PERP and short ETH-PERP is hedged: the account owes less than the two legs would owe separately. That offset is the product. But the two legs sit on different matching shards, written concurrently, and an order arrives at one of them. No single place sees the whole account at the moment the order has to be accepted or refused, and “requirement $\leq$ equity” is a statement about the whole account. Checking each book on its own lets an account build more than it can pay for: E2's naive control reaches a requirement of 201,000 on 2,000 of collateral.

Why not ask a central service on every order. Admission happens at N gateways in front of the shards. Either every order pays a round trip to one shared counter per account, or each gateway is handed a share it can check locally. **The value of the share is upstream early shedding** — stopping a burst before it queues at a single-writer shard. With one gateway the design collapses to a local counter, and we would say so rather than defend it.

How the requirement is split. The requirement has two parts.

- **The scenario part** is the worst loss the account takes over a fixed set of market moves. It *can* be split: the worst case of the whole account is never more than the worst cases of its parts added up, and equals that sum only when one market move is the worst for every part at once. So each gateway gets its own risk budget and checks it locally.
- **The concentration add-on** grows faster than size (it is convex). Pieces of it add up to *less* than the whole, so splitting it would under-charge. It is never split: it is reserved once, centrally, on the total size all gateways together may reach.

What the closure buys. The allocator issues budgets so that

$$2 \times (\text{risk budgets}) + \text{add-on}(\text{total size budget}) + (\text{cost budgets}) \leq \text{equity}$$

The **two** is exact, not a safety buffer. A filled position does two things to the account: it *owes* its requirement, up to the risk budget, and if the bad market move happens it *loses* money, also up to the risk budget, because the scenario part is by definition the worst loss. Both have to fit inside equity. With equity 100 and a risk budget of 50: fill to 50, the worst move loses 50, equity is 50 and the requirement is 50 — exactly equal. With a factor of 1.5 the budget is 66.7, the same move leaves equity at 33.3 against a requirement of 66.7, and the account is in breach.

The result: **the requirement stays inside equity after any market move the scenario set covers, with no allocator call per order.** The price: only about half of equity is usable. In E1's binding trial every order fills at the worst price and fee the policy allows; the budgets reach 99% and the requirement is 49% of equity, with no breach. Appendix D carries the algebra.

Model risk, stated. Most correctness experiments in §2–§5 use a simple scenario set: seven points on one factor. The split and the closure hold for *any* finite scenario set; E8 re-runs E1's oracle on seventeen scenarios with two factors, loadings of both signs and idiosyncratic moves, and finds no breach in 600 trials, including 300 driven to 99% of the risk budget, while a control that drops the factor of two breaches in 6 of 300. What is not shown is that any particular set is adequate for 40 underlyings — that is a calibration question. **When the set is wrong, the architecture does three things:** a move outside it is handled by liquidation and the insurance fund (§5.4), which is what E9's flash crash exercises; the set is versioned data on the log (§4.1), so it can be widened between sessions and a replay still reproduces old decisions; and a wider shock range costs capacity, while more scenarios cost some admission time, because admission is one pass over the set (E3: 7 → 16 scenarios, +34%); neither affects correctness.

2.3 What a lease grants

A lease is three budgets per account per gateway, for a term of 50–200 ms. **A lease cannot undo an admission it has already granted.** Everything below follows from that.

Budget	Bounds	Why it is separate
Risk	worst-case scenario loss of everything the gateway holds	the part that can be split
Size (gross)	the largest total size the gateway's orders can reach	an order can lower risk while raising size, and the add-on is charged on size
Cost	fees and slippage the gateway's orders can still incur	it lowers equity, and neither of the others covers it

The three are issued at fixed ratios, so the allocator searches one number per account, not three.

What a gateway checks is orders, not positions. Two resting orders of opposite sign net to nothing, but if only one fills the account carries the other side. So each budget is checked against the worst subset of fills that could still happen.

Because loss under one market move is linear, that needs no enumeration; the closed form matches brute force over every possible subset of fills on 4,000 random books.

Absolute figures, not the increment an order adds. An order that flips a leg from short to long leaves the increment unchanged while the account's requirement jumps to its maximum (c1). Admission compares the whole worst-case figure against the budget.

Size is reserved at the highest price the scenario set reaches. The add-on depends on size, size depends on price, and prices move during a term. In the worked case m1, reserving at today's price admits 296 lots and ends 382,143 above equity once the market reaches the edge of the set; reserving at the highest price admits 249 lots and ends 5,842 inside. The cost is 47 lots, about 16% of capacity (ADR-3).

The equity the budgets are solved against is mark-to-market equity, not collateral: collateral plus cash and position value at current marks, less fees. That is why the mark-price pipeline (§2.5) sets how much capacity exists.

2.4 Ending authority

Every lease carries (account, epoch, generation, lease_id) and is registered at the ordering point against the account, the holder and the authority kind (Appendix C.3). A holder is (gateway_id, incarnation): a process that restarts and reuses its identity is a different holder, and both may be live at once.

Two quantities are tracked per holder and only one expires. **A term ends authority to admit; it never ends the exposure already created.** A clock comparison does not prove a partitioned holder has stopped — its clock may be behind. A **fence at the ordering point** does, because nothing reaches a book except through it, and it need not reach the gateway: E7 fences without telling any gateway, the ordering point turns away 50 submissions, and the run is otherwise identical.

A fence is not terminal for exposure either: a resting order can still fill. Removing its reservation needs a cancel acknowledged at the ordering point; lowering a holder's recorded exposure needs a terminal reconciliation with that lease's seal, or an account-wide barrier. §5.4 is the full lifecycle.

When the solve is infeasible, leases are issued in **quarantine** and the gateway admits nothing — including orders that look locally like risk reduction, because an order lowering one gateway's requirement can raise the account's by removing a hedge held elsewhere (c9).

2.5 Mark prices

Marks carry no authority on the admission path — the check reads no market state — but they set mark-to-market equity, where the scenario set is centred, and the highest price size is reserved at. So **a wrong mark changes how much capacity is solved for**, and a mark held too high is §6.3 A1 by another route: E5 measures a breach that tracks an equity overstatement one for one.

Decision	What we get	What we pay
Several independent index sources per underlying, each checked for staleness on its own	one frozen or manipulated feed cannot set the mark	more feeds to operate and reconcile
The mark is a trimmed statistic across the live sources, with a bound on how far it may move per publish	a single outlier print is dropped	a genuine fast move is followed with a lag of a few publishes
Marks are versioned data on the log, activated at a sequence (§4.1)	every admission and liquidation replays against the mark it saw	a correction is a new record, not an edit
Fewer than a quorum of live sources: the allocator stops issuing and existing leases run out their term	capacity is never solved against a mark nobody can confirm	order entry for affected accounts stops within one term
Divergence across sources pages (§8.2, alert 4)	an operator sees the attack or the outage	—

The number of sources, the trim and the per-publish bound are operational parameters chosen per underlying; none of them is derived here. The pipeline is designed and not built: the simulator takes marks as given.

2.6 Components

Component	Holds	Decides
Ingress gateway	three ceilings per account; worst-fill running totals over the grid	admit or refuse, by comparing absolute figures against ceilings. Its state is a deterministic fold of the log, so a snapshot bounds replay time rather than being a correctness requirement
Ordering point	the log; the lease registry; sessions; fences and seals	that an admission is the next number for a live, correctly bound lease; that a fill matches its recorded terms; that a basket commits as one record. Writes nothing when it refuses
Margin allocator	committed exposure per holder; generations; credit versions	the condition of §2.2, once per account per issuance. Sharded by account, ≈ 16 shards
Liquidator	the merged account view	which basket to transfer, checked against the merged account rather than a ceiling — the check c9 shows a gateway cannot make. Inside the trusted computing base on its own account (§6.1)
Mark-price pipeline	marks, per source and published	nothing on the admission path, but marks set equity and the reserve, so this path fixes how much capacity is solved for (§2.5)
Matching core	books	unchanged; applies a client order ID at most once
Ledger	postings	double-entry, append-only. A lease never appears in it (§4.3)

2.7 What is on the replicated log

Every admission with its lease, sequence number, holder and terms; every fill with price and fee; every cancel acknowledgement; every fence with its seal; every liquidation basket as one record; every settlement barrier; and the lease inputs per account per issuance.

Three components reconstruct the same facts from it without reaching each other: a recovering gateway rebuilds its order state, the ledger rebuilds the account, and the allocator computes occupancy for a holder that may be gone.

Not on the log: the per-gateway ceilings, which cost ≈ 32 MB/s against an ≈ 12.8 MB/s order stream to carry a value each gateway derives from ≈ 8 MB/s of inputs (ADR-4 has the arithmetic). Also not on the log: the scenario vectors and running gross, caches of pure functions of the order state.

Batching. Replicating a batch of commands amortises the round trip (Part 4 §3). Two rules keep it compatible with this design: the ordering point numbers a batch at entry and refuses the whole batch on any gap, so a seal never covers a hole; and a fence takes effect between batches, never inside one. Neither is implemented; the simulator commits one admission at a time.

2.9 Architecture diagrams

The three views §2 requires — component, data flow, deployment — plus the degradation ladder. The liquidation flow is in Appendix E.

Figure 1 — Component view

Two authorities partitioned on different keys, and one ordering point that both of them and every recovery path read from. The liquidator is drawn apart from the gateways because its orders are checked against the merged account rather than a ceiling, and its transfers do not touch the order books.

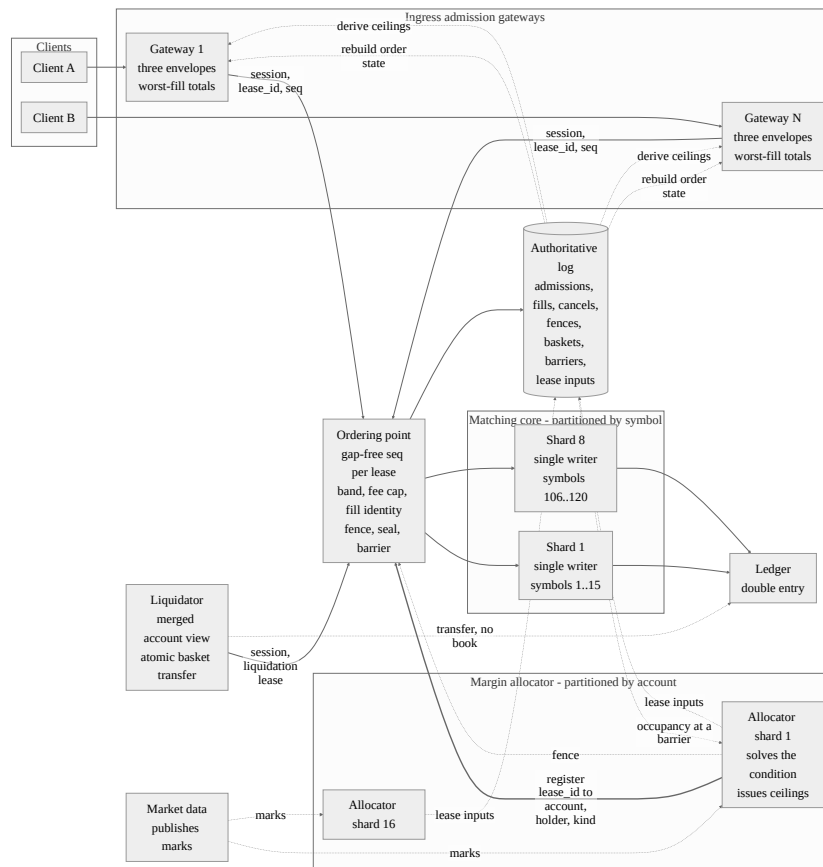


Figure 1: Component view. Two authorities partitioned on different keys, and one ordering point both of them and every recovery path read from. The thick edge is the lease registration this design would be unsound without.

Solid edges are the order path; dashed edges are asynchronous. The allocator never reads a gateway: every figure it acts on comes from the log. The thick edge is the one this design would be unsound without — a `lease_id` alone is a bearer token, and the ordering point knows which account, holder and authority kind it belongs to only because the single issuer registers the binding.

Figure 2 — Data flow for one order

Three stages. The first two are the gateway's and nothing downstream re-derives them; the third is the ordering point's and a compromised gateway cannot influence it. Refusals are collapsed to one node per stage.

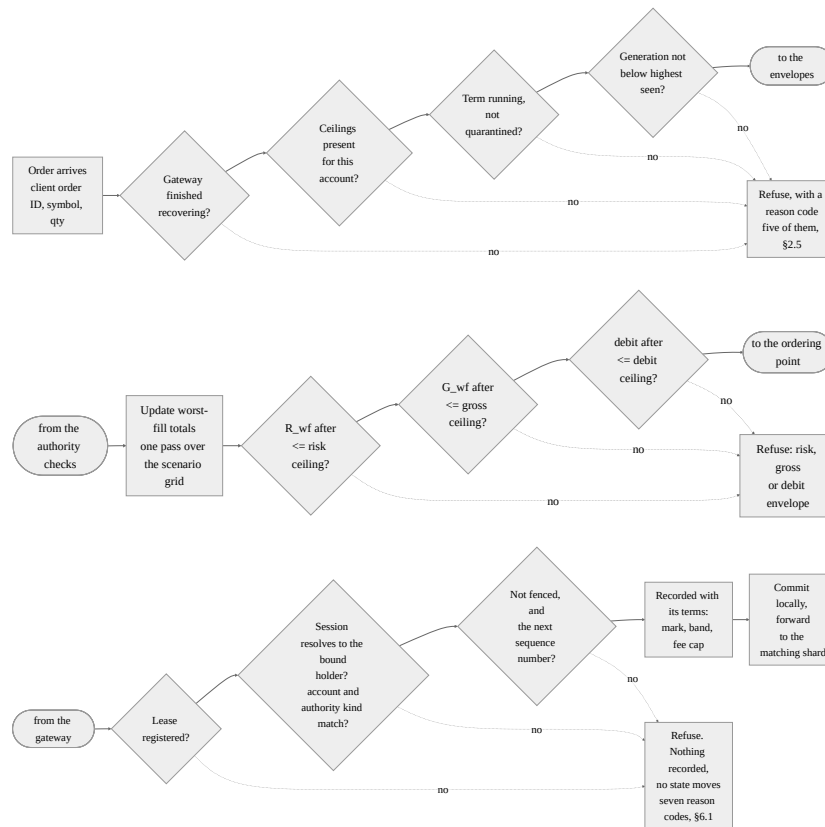


Figure 2: Data flow for one order, in three stages. The first two are the gateway's and nothing downstream re-derives them; the third is the ordering point's and a compromised gateway cannot influence it. Refusals are collapsed to one node per stage; the reason codes are listed in the sections cited.

The whole per-order margin cost is the one pass over the scenario grid in stage two: the running totals are updated per order state change rather than recomputed, so admission is flat in the order count and linear in the grid width (E3).

Figure 3 — Failure paths and the degradation ladder

Every transition is labelled with what causes it and what the venue still accepts in that state. A state a previous version called REDUCE_ONLY has been removed: a gateway does not accept locally-judged risk-reducing orders, because c9 shows that judgement is unsound across gateways.

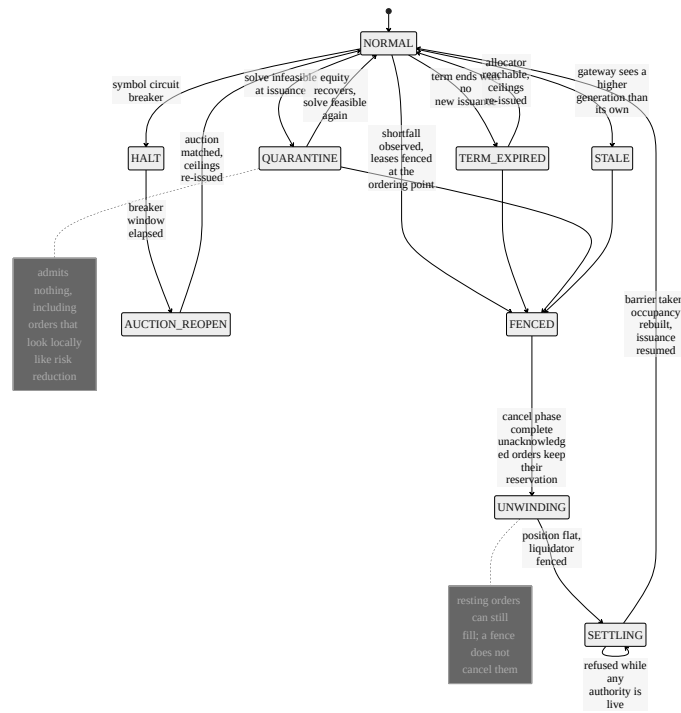


Figure 3: Failure paths and the degradation ladder. The four edges into **FENCED** are the four ways this design loses authority, and all four fail closed for new risk.

The four edges into **FENCED** are the four ways this design loses authority, and all four fail closed for new risk. Risk reduction happens on the liquidation path in every one of them, which is the CAP position §3.3 defends and the correction that removed **REDUCE_ONLY**.

Figure 4 — Deployment view: target, not built

Everything measured in this document runs in a single process against an in-memory ordering point. **No part of this figure has been implemented or measured.** It is drawn because the architecture is not complete without saying where each component lives and what replicates it, and it is labelled because a diagram of unbuilt infrastructure next to measured results otherwise invites the reader to give both the same standing.

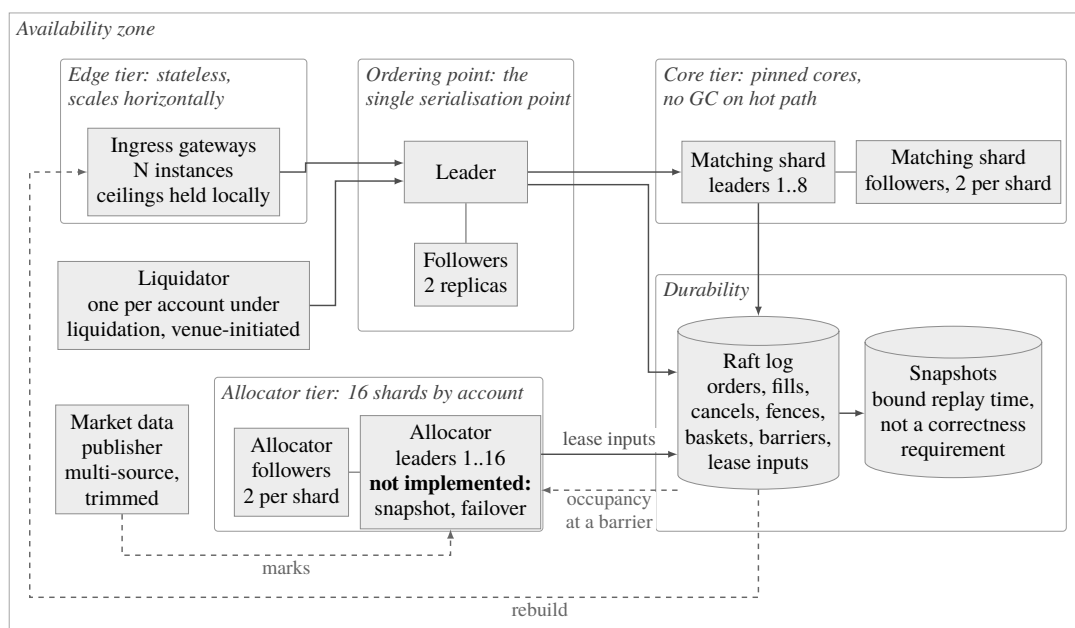


Figure 4: Deployment view. The target deployment; no part of it has been implemented or measured.

Three things here are load-bearing and unbuilt, and they are the three §5.7 lists. The ordering point is replicated, and every claim about fencing, sealing and barriers assumes it survives a node loss without losing or reordering the log. The allocator has no snapshot or failover: it is the only component that cannot rebuild itself from the log. And the liquidator is venue infrastructure, which is what makes its transfers internal (§5.4) and what puts it inside the trusted computing base (§6.1).

What the recovery evidence supports is the property replication needs — each component’s state is a deterministic, idempotent fold of one ordered log — and not that the replication works.

3. Consistency map

3.1 Per flow

One row per flow, with the model chosen and why it is survivable. The test is “weakest the business can survive”, not “strongest available”.

Flow	Model	Defence
Matching, per symbol	Single-writer total order	Price-time priority <i>is</i> a total order; anything weaker changes which client got the fill
Envelope consumption at a gateway	Local atomic within the gateway	Three integers and a vector owned by one gateway; nobody else reads them during the term
Admission sequence per lease	Gap-free total order at the ordering point	It accepts only the next number for a lease. The seal, the barrier and the settlement figure all rest on the log holding every admission that lease produced
Authority binding	Registry at the ordering point, holder from the authenticated session	A lease id alone is a bearer token: without the binding, knowing one is enough to submit for any account (Appendix C.3)
Fill terms	Checked at the ordering point against terms recorded at admission	A fill reported by a component that could be compromised is not evidence. Band, fee cap, direction, over-fill and identity are decided there; a refused fill writes nothing
Lease issuance and generation	Single allocator per account, linearisable	Two issuers could double-issue against the same equity. The one place the design cannot weaken
End of authority	Fence at the ordering point, linearisable	A clock comparison is not evidence a partitioned holder has stopped; a fence is. It need not be delivered to the holder
Marks into the allocator	Bounded-stale	Staleness costs capacity in both directions and, inside the grid, not safety — for two reasons that are both needed: the add-on is reserved against G^+ , the highest gross the grid reaches, and $\mathbb{R}$ already covers the equity the account loses at any scenario in it
Position feed into the allocator	Bounded-stale, eventually consistent	A fill the allocator has not seen was admitted under a lease and is inside that lease’s absolute ceilings. Until a terminal ordered reconciliation it stays charged to that holder
Holder occupancy	Over-approximated per holder while any is live; compacted from the log once none is	Per-holder figures are summed and do not net, which is necessary while an unreachable holder may still be acting (§5.4)
Audit journal	Durable append-only, linearisable per shard	Replay must reproduce the decision exactly, which fails if entries reorder
Dashboards and capacity displays	Don’t-care: eventually consistent, may lag	No decision depends on them; anything a client acts on is re-checked at admission
Balances that authorise money, such as a withdrawal	Strong: read only once the ledger’s fold has reached the account’s latest fill	Paying out against a stale balance could release money the account no longer has (§6.2)
Retry handling	At-least-once delivery with end-to-end idempotency	Exactly-once is not a transport property (§5.3)

The row worth arguing is the position feed. A stale feed cannot under-state the requirement, and the reason is **not** that lag produces a smaller ceiling — neither the mechanism nor reliably true. It is that the ceilings are absolute rather than incremental and the accounting is monotone: the allocator never releases a holder’s occupancy on the strength of not having heard from it.

3.2 CAP position of the matching core

Unchanged from the running case: during a partition that costs the leader its majority, the matching core stops accepting orders rather than risk divergence. Halting is embarrassing; double-executing trades is existential.

3.3 CAP position of the admission plane

The relevant partition is **between a gateway and the allocator, with the ordering point still reachable**. A gateway cut off from the ordering point cannot reach a book at all and is not serving in any sense.

During a gateway–allocator partition, the gateway keeps admitting inside the ceilings it already holds until its term ends, and then admits nothing. It does not fall back on a local judgement that an order reduces risk.

It cannot tell a risk-reducing order from one that closes a leg whose hedge sits on another gateway (c9), so it does not try.

Availability is bought before the partition. The gateway serves because its budget was solved to be safe on its own; the term length *is* the availability budget, and the same number as the tightening latency of §1.5. **When the term ends, all order flow through that gateway stops, closing orders included.** What survives a partition is the venue’s ability to bound its own loss, through the liquidator — not the client’s ability to exit. A client-facing reduce-only path would need the account-level check the liquidator makes, so a central component the gateway can reach; it is not built.

3.4 Where the design refuses to weaken

1. **One issuer per account.** Two allocators can issue two sets of ceilings against the same equity; each is individually valid, so no downstream check recovers.
2. **Authority ends at the ordering point, not on a clock.** If a fenced lease can still admit, the seal names a count that is already wrong and every figure computed from the log is computed over an incomplete one. E7’s `no_fence` arm draws 7,790 on the insurance fund against 0 in the base run, takes 75 transfers instead of 24, and cannot settle at all.
3. **Append-only, ordered audit.** Reordering makes an admitted decision unreproducible, which is the whole of what §6.4 offers a supervisor.

Everything else is chosen at the weakest model that survives.

4. Data and storage design

4.1 Engine per store, chosen from the access pattern

Store	Access pattern	Engine	Why not the obvious alternative
Order books	Top ticks at 100k/s; cancel by ID O(1)	In-memory price-level array, FIFO per level, index by order ID	A tree of queues costs a cache miss per hop; at 100 µs the misses are the budget
Scenario vectors and gross totals	Update per order, per (account, gateway)	Flat array, ≈ 64 MB resident	A revaluation per order is pointer-chasing at the top of book
Account positions and equity	Read per issuance; written on every fill	In-memory in the allocator shard, snapshotted	A row store serialises the hot accounts, which are the market makers
Log, journal and ledger entries	Sequential append at ≈ 21 MB/s (§2.7)	Append-only segments on NVMe	Nothing is updated in place
Balances	Fold of the journal, materialised	In-memory hash, snapshotted with the journal offset	Balances as source of truth is the running case’s anti-pattern, Part 3 §7
Historical balance-at-time-T	Rare, analytical	Columnar store fed by change data capture	Otherwise an analytical workload sits behind the trading lock
Scenario grid, add-on parameters, gateway weights, band and fee policy, credit version, authority bindings	Read on every derivation and replay	Versioned data on the log, activated at a sequence	As configuration they make replay non-deterministic

The last row is specific to this design: those values look like configuration an operator would tune, and are not, because a gateway derives its ceilings and its refusals from them and a replay must reproduce both.

4.2 Accounting model

Double-entry, with conservation enforced at write time: every journal entry's postings sum to zero per asset, so value is moved by code and never created. Accounts are keyed (ownerId, assetId, accountType) over USER_AVAILABLE, USER_MARGIN_HOLD, USER_UNREALISED, EXCHANGE_FEE, INSURANCE_FUND, EXCHANGE_HOT, EXCHANGE_COLD, SUSPENSE, EXTERNAL_SETTLEMENT, VENUE_BOOK and FUNDING_CLEARING. USER_MARGIN_HOLD is the running case's USER_HOLD under cross-margin: there the hold is per order and released when that order resolves, here it is the account's single encumbrance and moves with the portfolio requirement. Everything from USER_UNREALISED onward except the wallets and SUSPENSE is new relative to a spot venue: a derivatives venue carries positions it marks every tick, pays funding between clients, takes positions onto its own book in a liquidation, and keeps a fund to absorb what an account cannot.

The ledger is the venue's own books. Every posting is a debit or a credit, and an entry is valid only if its debits equal its credits in each asset. The venue's wallets, EXCHANGE_HOT and EXCHANGE_COLD, are assets and normally carry a debit balance. Every USER_ account is money the venue owes a client, so it is a liability with a normal credit balance, and the venue's own accounts, EXCHANGE_FEE, INSURANCE_FUND and VENUE_BOOK, are credit-normal as well. SUSPENSE, EXTERNAL_SETTLEMENT and FUNDING_CLEARING are transit accounts that return to zero. A client's equity is therefore the combined credit balance of USER_AVAILABLE, USER_MARGIN_HOLD and USER_UNREALISED, which matches the simulator's collateral plus PnL at marks, less fees. Paying out a withdrawal lowers a liability, which is a debit, and an asset, which is a credit, so one entry balances; an earlier version called this "two debits" and split it in two, which was a sign error rather than a fix.

Amounts are signed 64-bit integers in minimal units with 128-bit intermediates. No floats: they break determinism, which is what makes replicas agree and replays reproducible.

4.3 Leases, holds, and what actually follows

A lease is an authorisation. A hold is a posting. A lease never appears in the ledger: it is capacity to create holds, issued by the allocator, consumed by a gateway, ending with its term. A hold moves collateral from USER_AVAILABLE to USER_MARGIN_HOLD when a position opens. Conflating the two is how an authorisation becomes spendable.

An earlier version chained them into $\text{sum holds} \leq \text{sum consumed leases} \leq \text{sum issued leases} \leq \text{collateral}$. That is withdrawn and does not hold: orders are checked against absolute worst-fill envelopes rather than each consuming an additive quantity, so "consumed leases" is not a sum, and a scenario requirement, a gross notional and an execution cost are not commensurable and do not compose into a ledger amount.

What does hold is three facts that meet at the account: every posting sums to zero per asset, enforced at write time; an account's equity is a cash-flow fold of the authoritative log, rebuildable independently of any live component (a14); and the admission condition of §2.2 keeps the requirement inside *that* equity after any move the grid covers.

They do not compose into a proof that venue assets exceed liabilities: that needs the ledger implemented, the insurance fund sized against moves outside the grid (§5.4 argues a size from one stress run; it is not a calibration), and the venue book's own risk bounded.

4.4 The order lifecycle as journal entries

Each row is one balanced entry, written Dr (debit) / Cr (credit).

Event	Entry
Deposit confirmed	Dr EXCHANGE_HOT x / Cr USER_AVAILABLE x
Order admitted	None. Admission consumes a budget, which is not money
Fill	Dr USER_AVAILABLE / Cr USER_MARGIN_HOLD for the rise in requirement; fee Dr USER_AVAILABLE f / Cr EXCHANGE_FEE f
Mark-to-market	Dr loser's USER_UNREALISED u / Cr winner's USER_UNREALISED u; VENUE_BOOK takes the same entries for positions it holds
Position reduced	A gain moves from unrealised to available, Dr USER_UNREALISED / Cr USER_AVAILABLE (reversed for a loss), so it is never counted twice; released margin Dr USER_MARGIN_HOLD / Cr USER_AVAILABLE
Funding	Payer Dr USER_AVAILABLE p / Cr FUNDING_CLEARING p; receiver Dr FUNDING_CLEARING p / Cr USER_AVAILABLE p. The clearing account ends each interval at zero, so the venue is not a party
Liquidation basket	One entry per basket: unrealised PnL is realised into USER_AVAILABLE and the position passes to VENUE_BOOK at the basket price. A negative ending equity is covered Dr INSURANCE_FUND d / Cr USER_AVAILABLE d, bringing the account to zero

Event	Entry
ADL	An opposing profitable position is closed at the bankruptcy price instead of the mark: Dr its <code>USER_UNREALISED a</code> / Cr the defaulted account's <code>USER_AVAILABLE a</code> , one entry per event
Withdrawal requested	Dr <code>USER_AVAILABLE x</code> / Cr <code>SUSPENSE x</code> , after the sequence in §6.2
Withdrawal sent	Dr <code>EXTERNAL_SETTLEMENT x</code> / Cr <code>EXCHANGE_HOT x</code>
Withdrawal confirmed	Dr <code>SUSPENSE x</code> / Cr <code>EXTERNAL_SETTLEMENT x</code> . If the chain rejects the transfer, the sent entry is reversed

The funding interval, the rate formula and the ADL ranking are venue policy and are not specified here.

The order-admitted row is the one that matters: admission moves nothing, which is what lets it run at gateway speed, and why the envelope bound has to be sound — it is the only thing between an admitted order and an over-committed balance.

A withdrawal reduces equity, so capacity outstanding against the old figure has to stop before funds leave: **fence, reconcile, re-issue against the reduced equity, release**. Bumping the generation is not sufficient — a partitioned gateway keeps admitting inside its term regardless. §6.2 gives the slower alternative.

4.5 Write path, idempotency and reconciliation

Entry IDs are a hash of the trade sequence and leg index, so a replayed trade deduplicates. Assert-then-apply is atomic over the accounts touched; the ledger is a single-writer partition per asset class. A client-initiated entry, such as a withdrawal, may not take `USER_AVAILABLE` below zero, and that is asserted at write time. Entries the venue posts, such as marks, funding and liquidation, may leave equity negative; that is the shortfall the liquidation entry covers from the insurance fund. Reconciliation runs continuously: balances recomputed from the journal against the materialised view, sequence-range checks for gaps and duplicates, hot and cold mirrors against chain balances, and every account rebuilt from the log against the live ledger — which catches divergence between §4.3's two folds.

The ledger module is specified here and not implemented in the simulator, which models the envelope and account sides only: §4.3's three facts are established for the account, and NFR row 11's venue-level statement is an argument, not a checked property.

5. Failure and recovery

5.1 What has to survive a node loss

State	Owner	Size at the scale of §1.1	Rebuilt from
Order books	Matching shard	≈ 160 MB per shard at 8 shards	Snapshot plus log tail
Per-account positions, equity, ceilings, generation	Allocator shard	≈ 27 MB total	Snapshot plus log tail. Not implemented (§5.7)
Per-(account, gateway) scenario vectors and gross totals	Gateway	≈ 64 MB total	Derived; not snapshotted
Balances	Ledger	≈ 100 MB	Journal fold

The scenario vectors are a cache of a pure function of the order state, so they are rebuilt rather than restored: reconstructing 128 bytes from a handful of orders is faster than reading it from disk, and snapshotting them would double the snapshot and buy nothing.

5.2 What must be on the replicated log

§2.7 lists what an admission decision's replayability puts on the log and why the ceilings are derived rather than logged. The requirement is stronger than the running case's because three components must reconstruct the same facts without reaching each other, and two consequences carry into the rest of this section: **a snapshot bounds replay time and is never a correctness requirement**, and **replay is idempotent by log position** rather than by order ID — which covers admissions but not fills or cancels, and r2 caught that.

5.3 The idempotency chain

The full nine-step chain is Appendix C.1. Its shape:

- **the client** retries on the same client order ID; a timeout is an unknown outcome, not a failure;
- **the gateway** submits under `(lease_id, admission_seq)`, and the ordering point takes only the next number for that lease, so the recorded sequence is gap-free — which is what later lets a seal cover every admission;
- **the ordering point decides first**. Only a fill it accepted is folded into the gateway and the account; an earlier implementation called the gateway first and moved state the authority then refused;

- **a fill** is refused, writing nothing, on a reused identifier with different figures, an unknown or cancelled order, the wrong direction, an over-fill, a price outside the recorded band, or a fee above the cap; **a basket** is one record under one identifier, idempotent on retry; **the matching shard** applies a client order ID at most once.

Two costs stated rather than hidden. A client retry, through the same gateway or another, is a new admission attempt and holds envelope until the duplicate's rejection is recorded; a new issuance does not release it. And a cancel *request* releases nothing; only an acknowledgement recorded at the ordering point does, which produces the two failures in Appendix E.2.

5.4 Fencing, liquidation and settlement

An account cannot reach a requirement above its equity through any move the grid covers; that is what the closure reserves for. **A liquidation is therefore an event outside the model, the credit event has already happened by the time anyone sees it, and the mechanism is limiting a loss rather than preventing a violation.** What it limits is the draw on the insurance fund once the account's own equity is gone.

Three things stop, in three places

What	Where	Cost
New admissions	a fence at the ordering point	one write; no gateway need be reachable
Resting orders filling	a cancel acknowledged at the ordering point	a round trip per order, and it can fail
The position moving	trading it out	as long as the unwind takes

The unwind is an internal transfer Reducing one leg at a time does not work: on a hedged book, closing one leg while its offset stays put raises the account's requirement — c9 with the liquidator in the gateway's place. 13 measures the requirement going $0 \rightarrow 2,000$ on a single-leg proposal and staying at 0 on a proportional basket.

That forces a basket, and matching is sharded by symbol, so **a basket spanning several symbols cannot fill atomically across several books**; a partial fill on one shard with none on another leaves the account somewhere the check never approved. This design does not assume an atomicity the matching path lacks and does not route baskets through it. The liquidator prices the whole basket against the venue's own marks, inside the same band and fee cap an ordinary fill is held to, and the ordering point commits it as one record: either the record is there and the whole basket happened, or none of it did.

The cost is that the venue is the counterparty. Risk moves to the venue's own book and, past the account's equity, to the insurance fund. What limits that is below, under the waterfall.

What a cancel and a delay cost A cancel releases an order only once its acknowledgement is recorded at the ordering point. An order the matching side never confirmed keeps its reservation, and fencing does not help, because a fence stops new admissions and not resting orders. Delay is bounded only in part: the mechanism bounds new admissions and the cost of unwinding, but not market drift, which in E6 reaches 229,708 against execution costs of at most 6,486. Appendix E.2 gives both cases with E6's and E7's figures.

Releasing capacity afterwards **A seal releases one lease; a globally fenced, ordered account barrier releases the whole account.** Neither takes a holder's word or a clock: both read the log. The barrier has six preconditions, none supplied by the caller (Appendix C.2); E7 refuses it while any lease — including the liquidator's — is live.

Below the unwind: venue book, insurance fund, ADL Designed, not built. Each layer is a decision with a price.

Layer	Decision	What it costs
Venue book	The venue's own book is margined like an account, against capital the venue allocates to it. It unwinds through ordinary matching, one symbol at a time, with a cap on its share of each book's volume	Slow exits in a thin market. A transfer that would take the book over its limit is not taken; that part goes straight to the next layer
Insurance fund	Covers an account's negative ending equity, from the E6 identity. Sized to cover the combined shortfall of the largest accounts under stress moves beyond the scenario set, plus a floor it may not be drawn below	Capital held idle. How many accounts and which stress moves are policy; we give the method and one run, not a calibration

Layer	Decision	What it costs
ADL	When a draw would take the fund below its floor, the rest is taken from opposing profitable positions at the bankruptcy price, one record per event	Profitable clients lose part of a position they did not choose to close. It is the last resort because it is the only layer that touches clients who did nothing wrong

A flash crash Session 3's timeline (slide 29) is our test vector. The table maps each event to MarginStream's response and whether it is tested or design only; since the slide's timings are illustrative, E9 keeps its own clock in ticks.

Time	Event	MarginStream response	Evidence
T+0s	Price -4% elsewhere	Mark follows within a few publishes. 4% is inside the scenario set (20% on symbol A), so budgets stay safe and nothing is liquidated	Tested (E1, E8). Mark pipeline: design only
T+2s	Orders ×20, cancels ×40	Admission is local and flat in live orders. ≈ 2M/s exceeds the 1M/s burst target, so gateways shed the excess (T+8s). A cancel frees budget only once recorded at the ordering point	Tested: scaling (E3), cancels (E7). Throughput: not measured
T+5s	Market-data fan-out saturates	Client data is conflated; the allocator's mark feed is separate and never conflated	Design only
T+8s	Retry storm	Gateways accept up to their share of the 1M/s burst and refuse the rest with a jittered back-off hint; a refusal changes no state. A gateway resending the same submission is applied once at the ordering point. A client retry, at any gateway, is a new admission that holds budget until its rejection is recorded, and matching applies the client order ID at most once	Tested: gateway resend. Client-retry dedup: cited, not simulated. Shedding: design only
T+12s	Liquidations sell into thin book	The venue takes positions in atomic basket transfers at the mark (half per tick in E9), keeping forced selling out of the book. Selling is deferred, not removed: the venue book unwinds later under a volume cap	Tested: transfers (E9), cost identity (E6). Venue-book unwind: design only
T+15s	Breaker, 10% / 60 s	Trips at half the widest scenario step: 10% on symbol A (4% on B, whose range is narrower). Halts matching, admissions and the liquidator	Tested (E9): threshold. Duration: not modelled
T+45s	Call-auction reopen	Budgets re-issued at the auction price. E9 assumes two reopen prices, 60% recovered and at the low	Tested: liquidation at assumed price. Re-issuance, auction: design only
T+60s	Clearing 40 s behind	Balances lag but stay correct as a fold of the log. Admission never reads them; a withdrawal waits for the ledger to catch up	Design only

A 4% fall inside the scenario set triggers no liquidation and so tests nothing below the account. E9 therefore drives a far larger fall, 3.4 widest scenario steps (68% on symbol A), twice: as a **gap** over three ticks and as a **slide** over ten. Five long accounts with 950,000 of collateral between them, a fund of 100,000 and a floor of 20,000. Two reopen prices are run, because the value of a halt depends on where the auction clears and the run cannot know that.

Crash	Breaker, auction recovers 60%	Breaker, auction at the low	No breaker
Gap, 3 ticks	no liquidation, no draw	draw 214,848; fund to floor; ADL 134,848	draw 214,848; ADL 134,848
Slide, 10 ticks	no liquidation, no draw	draw 214,856; ADL 134,856	liquidation keeps up; no draw

Three readings. Against a **gap**, liquidation cannot help: prices jump past the point where equity runs out between two chances to act, and only a recovering auction saves the fund. Against a **slide**, liquidation alone keeps up — and a breaker that halts early and reopens at the low makes it *worse*, because a halt stops the liquidator too. So **a breaker is a bet on the reopen price**, and its threshold should sit where gradual liquidation stops keeping up, not at the first sign of a move. And in this configuration a fund of about 235,000 — a quarter of the collateral — would have avoided ADL in every arm; that is one run’s figure, not a sizing rule. The E6 identity holds in every liquidated account.

The simulator measures neither throughput nor latency under the $\times 20$ load; the overload rows above are design, not results.

5.5 Recovery-time arithmetic

State is not one snapshot: the tiers of Figure 4 fail independently and are sized separately.

Tier	Snapshot	Log tail over 5 min	Conclusion
Matching shard	≈ 160 MB (one shard of eight)	orders and fills for that shard’s symbols	Dominated by the snapshot read, then replay
Allocator shard	≈ 27 MB total across sixteen shards	lease inputs, ≈ 8 MB/s	Small enough that replay dominates
Gateway	none — its state is derived (§5.1)	admissions, fills, cancels for its own lease	Rebuild is pure replay

The one figure that can be checked from this document is the replay volume:

5 min at ≈ 21 MB/s (§2.7) = 6.3 GB;
at an assumed $10\times$ live replay rate, the log tail takes about 30 s,
excluding snapshot fetch and leader election.

The 21 MB/s is a lower bound that excludes fills, cancels, fences, baskets, framing and replication, and the $10\times$ replay rate is assumed, not measured (§8.1).

Warm failover is a design target, not a result. Neither replication nor leader election is implemented (§5.7). What *is* established is the property failover needs: E4’s 3,642 injected crashes show snapshot plus replay reproducing the state the whole log implies, zero equivalence failures, including 2,364 mid-partial-fill and 819 stale-snapshot cases.

When an allocator shard fails over, outstanding leases and terms (designed, not built):

1. Leases already issued stay valid until their term ends. Gateways keep admitting inside them; nothing about a failover makes an issued budget unsafe.
2. The exposure those leases created stays charged to its holders. The new leader rebuilds occupancy per holder from the log, as the barrier does, and never from what a gateway reports.
3. Before issuing anything, the new leader commits an **epoch fence** at the ordering point: registrations from an older epoch are refused. A deposed leader that is still running cannot mint a lease, so there is still one issuer per account. Admissions under already-registered leases are not refused.
4. It then issues a new generation for every account on the shard.

The arithmetic, per account on the failed shard:

no-ingress window $\approx$ election + catch-up + one issuance - remaining term
cold rebuild: 5 min of lease inputs at an assumed $10\times$ replay ≈ 30 s
hot follower applying the log continuously: catch-up $\approx$ replication lag

Row 8’s three seconds is reachable only with hot followers; a cold rebuild misses it by an order of magnitude. And because a term (50–200 ms) is shorter than any failover, **the accounts on that shard — one in sixteen — lose order entry for up to the failover time**. Avoiding that means terms longer than a failover, which makes a credit cut under partition take seconds instead of milliseconds. We keep the short term and accept the gap: stopping order entry for 3 s on a sixteenth of accounts is cheaper than letting a cut credit line keep trading for 3 s.

5.6 Zero-downtime upgrade

Anything a decision is derived from is versioned data on the log, activated at a sequence (§4.1), so a replay uses the values in force then. Upgrades are rolling with the state machine version pinned per record, and an operator cannot tune a derivation mid-session — a real operational loss, and the price of NFR row 10.

5.7 What is not covered here

This is a deterministic simulator, not a deployment. Everything measured runs in one process against an in-memory ordering point. A replicated ordering point, allocator high availability, leader election and real distributed deployment are *designed* and *not built* (Figure 4); the recovery results establish the property replication would need, not that replication works. The allocator alone cannot rebuild itself from the log, and that is not implemented either.

The waterfall below the unwind, the allocator failover protocol and the mark-price pipeline are designed and not built; E9 exercises the waterfall's arithmetic in simulation. The replay rate of §5.5 is assumed, and cross-datacentre replication is not covered. Two overload responses in §5.4 are designed and not built either: gateways shedding orders beyond their share of the burst with a back-off hint, and withdrawals waiting until the ledger has caught up with the account's latest fill.

6. Security and threat model

6.1 Trust boundaries and the trusted computing base

Boundary	If the far side is fully compromised
Client ↔ gateway	The account's own ceilings. Ceilings are per account, so one client cannot consume another's
Gateway ↔ ordering point	Inside the trusted computing base. See below
Ordering point ↔ everything	Total. Every safety claim rests on it
Liquidator ↔ ordering point	The account it is liquidating, in full
Market data ↔ allocator	How much capacity is solved for. Not the admission check itself
Allocator ↔ gateway	An account's whole capacity
Chain ↔ custody	Venue assets. Unchanged from the running case
Operator ↔ ledger	Nothing; no write path. Detection is reconciliation, not prevention

The gateway and the liquidator both sit inside the trusted computing base. Nothing downstream recomputes a gateway's envelope arithmetic, so a gateway that lies about it is believed. Tests show that it still cannot act under a fenced or unregistered lease, use another account's or another holder's lease, commit a liquidation basket, or record a fill outside the terms set at admission. It is, however, bounded by neither its budgets nor its term, because the ordering point has no clock with which to enforce a term, and only a fence at the ordering point stops it. The mechanism therefore limits which accounts a compromised gateway can touch but not how much damage it can do. Closing that gap would put the margin calculation back on the single-writer path, which is what §2.2 exists to avoid. The liquidator is checked against the whole account rather than a budget, so it cannot raise the account's risk but can churn it, and the barrier refuses to settle while it is live. Appendix F gives the full argument and the test behind each claim.

6.2 Who can move money

Three ways, none of them the margin path: a trade, journalled by clearing; a deposit or withdrawal, frozen then reviewed then gated then signed once per withdrawal ID; and a liquidation or insurance-fund draw, journalled like any other posting. No component writes a balance. The ceremony scales with the amount: withdrawals above a threshold need two-person approval, every operator action is itself a journalled, dual-approved command, and cold-wallet signing is m-of-n on HSMs, so no single operator and no compromised gateway can move funds.

A withdrawal reduces equity, so outstanding capacity against the old figure must stop before funds leave. Two orderings work and differ in latency, not safety: fence, reconcile, re-issue, release — immediate, at the cost of a fence round trip; or wait for the terms to end, then re-issue and release — free, binds within the term. Releasing first is not available.

6.3 Business-logic abuse cases

A1 — Overstating equity Every ceiling is solved against E_0 , and nothing downstream re-derives it. An account reporting more equity than it has buys a ceiling it cannot carry.

E5 measures it. An account that forgets a realised loss reports 92,000 where it has 42,000, is issued 46,000 instead of 21,000, admits 230 orders instead of 105, and ends 4,000 above equity. At the binding point the breach tracks the overstatement approximately one for one, subject to lot rounding: overstating by 10,000 produces a breach of 10,000, while 1,000 produces 800, because the ceiling cannot move until the overstatement buys a whole lot of requirement. **The factor of two is a closure, not a margin against a misreported account;** an earlier draft that read it as a 64% tolerance was reading unused workload slack.

The exposure is to anything that moves E_0 : a suppressed mark, a lost fee, a realised loss not yet folded in. The defences are §4.3's cash-flow identity, the account being an independently rebuildable fold of the log, and multi-sourced marks. None is a proof; a compromised equity path is a compromised mechanism.

A2 — Routing to concentrate an account’s flow A gateway gets no credit for offsets held on other gateways, so a client who can steer hedged legs onto one gateway — by picking it, or retrying until it lands there — gets more usable capacity than one who cannot. A **fairness** problem, not a solvency one: the requirement stays bounded. Account affinity would narrow it; not built.

A3 — Stalling a liquidation Orders whose cancels the matching side never acknowledges keep the account fenced indefinitely: the settlement reserves for them (113). The venue’s exposure is bounded by that reservation, but the account cannot be closed out. Detection is §8.2’s fenced-but-not-settled alert; there is no mitigation. Fee-cap gaming through many small fills is bounded by the per-lot cap (d4).

6.4 Regulatory posture

A supervisor gets **reproducibility of what was admitted**: every admission is on the log with its lease, holder and terms, and the values it was derived from are versioned there (§4.1), so a three-month-old decision can be recomputed. Not yet for a **refusal**: a gateway’s refusal reaches no log (NFR row 10). And not a guarantee that clients can always exit (§3.3) — a supervisor asking should be told no.

6.5 Detection

Six signals, all paged on and listed in §8.2: equity divergence between what the allocator solved against and what a rebuild from the log implies; ceilings summing above the solve; a gateway’s figures above its ceilings; a lease fenced but not settled beyond a bound; mark divergence across sources; and any authority-binding refusal, which should be zero. The gap this list cannot close is §6.1’s: nothing detects a gateway that lies about arithmetic nobody re-derives, until the account’s own figures move.

7. Trade-offs and alternatives

Each decision with the alternatives considered and the reason each lost. Where a number decided it, the number is given. The reversals are summarised here and recorded in full in Appendix A.

ADR-1 — How the account-level invariant is enforced pre-trade

Decision. Divide the account’s capacity into per-gateway ceilings checked locally against absolute figures.

Alternative	Why it lost
Lock the account for the check	The only <i>exactly</i> correct option — full offset, no conservatism — and it puts the running case’s fee-account hot-row problem (Part 3 §4) on every order. Market-maker accounts are the hottest objects in the venue
Fixed per-gateway sub-limits, no offset	Safe, trivial, and it deletes the product. The value forgone is exactly the sub-additivity gap $\sum_g R(P_g) - R(P)$ taken to its maximum
Admit optimistically, repair after	Violates the rule that a balance never goes negative. A venue proving assets $\geq$ liabilities at any instant cannot have a window where the proof is pending

Cost. A gateway gets no credit for offsets held elsewhere, so the account is charged the sub-additivity gap. §6.3 A3 records the routing distortion that creates.

ADR-2 — Flat ceiling for the term, or a schedule over market states

Decision. A flat ceiling. **This reverses an earlier decision**, and the reason is short: a lease cannot remove a position it has already admitted, so capacity that shrinks with the market restricts the *next* admission and does nothing about the exposure already created. A flat ceiling at the solved level is equally safe and admits at least as many orders as any decaying one with the same start. The figures that argued otherwise came from a mechanism that charged the increment rather than the absolute envelope (c1), and are withdrawn with it. Appendix A.1 has the full reversal; Appendix A.2 keeps the abuse cases the schedule created.

What survives. A gateway evaluating a shrinking curve can notice locally, on a market-state tick with no order present, that its consumption is higher than the venue would like. That is a trigger, not a capacity mechanism, and its value is unmeasured.

ADR-3 — Where size is measured

Decision. Reserve size at the highest price the scenario set reaches, not at today’s price. **Alternative lost** on m1: 296 lots admitted, 382,143 over equity after the move, against 249 lots and 5,842 inside. **Cost:** about 16% of capacity. With one

factor the reserve is tight to a minor unit per lot; with several factors it stays safe but is loose (m4b: 24,000 against 20,000; E8: up to 8%).

ADR-4 — What goes on the replicated log

Decision. The lease inputs, one record per changed account per issuance; each gateway derives its own budgets. **Alternative lost** on bandwidth: logging the budgets costs ≈ 32 MB/s against an order stream of ≈ 12.8 MB/s; the inputs are ≈ 8 MB/s. **Cost:** everything in the derivation is versioned data (§4.1) and cannot be tuned mid-session.

ADR-5 — How the allocator is partitioned

Decision. By account, sixteen shards. **Alternative lost:** by symbol, like matching, would need several shards to combine partial views of one account — the coordination the design exists to remove.

ADR-6 — What ends a holder's authority

Decision. A fence at the ordering point, with the term as the fallback, and each lease bound to an account, a holder and a kind.

Alternative	Why it lost
Compare the order's generation with the lease's	a stale gateway and a stale order agree with each other
The allocator's clock against the expiry	a partitioned gateway's clock may be behind (c11)
The holder reports it has stopped	the defect that kept recurring until release read the log itself
The lease id as the authority	a bearer token: any account, any holder (t1)

Cost. The ordering point has no clock, so it does not enforce terms: an honest gateway is bounded by its term, a compromised one only by the fence (§6.1).

What the counterexamples forced

Every row is a failing test written first, then a design change. Detail is in Appendix A; the tests are in `tests/`.

Counterexample	What broke	Design change it forced
c1	charging an order its increment: a leg flipped from short to long left the increment unchanged while the requirement hit its maximum	gateways check absolute worst-fill figures
c9	a gateway accepting "risk-reducing" orders during a partition: closing one leg raised the account's requirement because the hedge sat on another gateway	a cut-off gateway admits nothing; only the liquidator, with the merged account, reduces risk
m1	size reserved at today's price: 296 lots, ending 382,143 over equity after the move	size reserved at the highest price in the scenario set: 249 lots, inside by 5,842
l3	unwinding one leg at a time: requirement 0 $\rightarrow$ 2,000	proportional basket, committed as one record, venue as counterparty
c8, c11, c12	exposure released on expiry, on a clock, or on the holder's own report	only a seal or an account-wide barrier releases exposure
t1	a valid lease id under another account returned success	the ordering point binds each lease to account, holder and kind
r2	replay de-duplicated by order id applied fills twice	replay is idempotent by log position
d7	capacity shrank every term	the cost budget carries cost already incurred but not yet in equity
E5 Part B	reading the factor of two as a 64% tolerance for misreported equity	the two is a closure; at the binding point a breach tracks an overstatement one for one
ADR-2	a budget that shrinks as the market moves, claimed as a safety mechanism	a flat budget per term; the schedule survives only as a local trigger

What we deliberately did not build

- **The waterfall, the mark-price pipeline and allocator failover** are designed (§5.4, §2.5, §5.5) and not built. E9 runs the waterfall's arithmetic.
- **A client-facing reduce-only path.** §3.3 says why, and what it costs.
- **Replication of the ordering point.** Figure 4 draws the target deployment and labels it unbuilt.
- **Anything that identifies the agent behind an order.** The mechanism is capacity control, not surveillance.

8. Operations

8.1 The first chaos experiments

Measure the replay rate, because §5.5 assumes it at ten times live and nothing else in that arithmetic is assumed. Cold-start a node from a snapshot plus five minutes of log, time the rebuild, and compare the state hash against the live node — the comparison E4 makes 3,642 times. A slower rate only shortens the snapshot cadence, since a snapshot bounds replay time, not correctness.

Second: partition a gateway from the allocator while it stays connected to the ordering point, and hold it past its term. This is the failure §3.3 is about. What should be observable is the gateway admitting inside its ceilings until the term ends and then admitting nothing at all, closing orders included.

Third: fence an account's leases without telling any gateway and confirm the ordering point refuses what follows. E7 counts 50 refusals in simulation; in a deployment this checks that the fence is a real serialisation point.

8.2 Alerts, business invariants first

1. **Sum of issued ceilings above what the condition solved for.** An arithmetic impossibility if the allocator is correct, so firing means it is wrong.
2. **A gateway's worst-fill figures above the ceilings it holds.** Same class.
3. **Requirement above equity outside a liquidation window.** The invariant the design exists to hold.
4. **Equity divergence** between what the allocator solved against and what a rebuild from the log implies, and **mark divergence** across sources. §6.3 A1 is the reason both page rather than sit on a dashboard.
5. **A lease fenced but not settled beyond a bound**, and any **authority-binding refusal**. The first is a stuck liquidation holding capacity; the second should be zero in normal operation and otherwise means a component is submitting under a lease that is not its own.

Alerts 1 to 3 are the invariants the simulator checks after every admitted order, promoted to production: the same predicate gates a test run and the live system, so a production failure is expressible as a failing test case. Latency percentiles, queue depths and replica lag are diagnostics and do not page.

8.3 Volatility-day playbook

Before. Pre-scale gateways; the matching core is partitioned, not elastic. Confirm every mark source is live and independent. Decide the term for the session: it is the tightening latency the venue accepts under partition (§1.5), trading availability against how fast a credit decision binds.

During. Nothing to tune. A tightening is a re-issue against lower equity, binding at the next issuance for reachable gateways and within the term for unreachable ones. To bind faster, fence the affected leases.

Liquidation is venue-initiated. The operator decides when to trigger, how aggressively to unwind, and whether to accept a stall: the unwind halves its basket fraction on a failed check and stops at one lot rather than forcing a reduction that would raise the requirement.

Halt and reopen. Cancels accepted, no new orders. On reopen, ceilings are re-issued against post-auction marks before order entry resumes.

After. Reconcile per §4.5, and confirm every account that entered liquidation reached a barrier: fenced but never settled is the state that silently costs capacity.

8.4 What operations cannot do

Operations cannot grant capacity by hand, recall what an unreachable gateway has already admitted, release an account's occupancy without a barrier, or write to the ledger, where corrections are journalled, dual-approved commands. The middle two were tried as manual overrides and produced c8, c11 and c12: an operator who could declare a holder finished handed its capacity to a replacement while the holder was still trading.

9. AI-disclosure appendix

9.1 How AI was used

An AI assistant was used throughout as a co-author: proposing mechanism, writing the simulator, deriving the Fermi estimates, drafting these sections. Every experiment was executed rather than described, and because the simulator is integer-only and seeded, runs on Python 3.12.3/Linux and 3.13.5/macOS agree except for E3's wall-clock figures (`results/PROVENANCE.md`).

Below is what the rubric asks for: what was rejected, and why.

9.2 Prompts that mattered

“Verify before building on it.” Applied to citations and to the assistant's own account of the design's risks; it surfaced that the risk named first was not the real one (9.3, item 1).

“Comments say what the code does, not what it is supposed to prove.” Applied to the simulator and every experiment script.

“Does the constraint actually bind?” Asked of the first E1 run, which reported zero violations because the budget was far above what the flow consumed. A zero from a slack constraint says nothing. E1 now carries a **binding trial** where the risk and debit envelopes reach 99% and the requirement is 49% of equity; only that zero is meaningful.

“Attack your own interface.” External hostile review was run each round against the code rather than the prose. It produced the authority-binding failure (9.3, item 6) and, late on, the discovery that a superseded experiment left on disk was passing on an empty sample.

9.3 Suggestions and claims rejected, and why

1. **That tiered maintenance margin breaks the decomposition.** Proposed as the leading risk. It is not: tiers are per contract and additive. What actually breaks decomposition is the portfolio-level add-on, which is super-additive.
2. **Abstracting the requirement into a general coherent-risk-measure formulation.** Rejected: Appendix D.1 rests on exchanging a maximum with a sum, and burying that step inside a general functional makes the paper harder to defend, not stronger.
3. **Charging an order its marginal requirement.** A leg flipped from short to long leaves the increment unchanged while the account's requirement moves to its maximum (c1). Replaced by absolute worst-fill envelopes.
4. **A price-conditional schedule as a safety and capacity mechanism.** Withdrawn with its evidence (ADR-2). A lease cannot remove a position it already admitted.
5. **Letting a gateway accept locally risk-reducing orders under partition.** Withdrawn: c9. Risk reduction is an account-level operation.
6. **Treating a lease id as authority.** Review demonstrated a valid lease id submitted under a different account returning success. Replaced by the registry and session binding of Appendix C.3.
7. **Reading the factor of two as a 64% tolerance for a misreported account.** Measured in a non-binding configuration. At the binding point the breach tracks the overstatement one for one (E5 Part B).
8. **Expiry releasing exposure, and a holder's own report releasing it.** Both replaced by seals and the account barrier (c8, c11, c12).
9. **That a circuit breaker always reduces the draw on the insurance fund.** E9 falsified it: against a slide, a breaker that reopens at the low turns a draw of 0 into 214,856, because the halt stops the liquidator too (§5.4).
10. **Keeping the algebra in the body.** After the mid-point check-in the body was rewritten around decisions, costs and risks, and the lemmas and the closure proof moved to Appendix D.

9.4 What the numbers are, and are not

- **Current evidence:** E1–E9 and eleven test files — 16 lifecycle counterexamples, 6 worst-fill cases plus an exhaustive closed-form comparison over 4,000 books, 6 recovery, 14 liquidation, 6 repricing, 6 authority, 7 execution-cost and 14 ledger cases, and a fuzz over 10,060 admissions.
- E3's absolute latencies support a **scaling** claim, not NFR row 2's target.
- E6's required-buffer figures are one configuration, one seed, one price path.
- The ledger of §4 is designed and not implemented; §4.3 establishes three facts about the account, not venue-level solvency.
- The replay rate of §5.5 is assumed, not measured (§8.1).
- §6.3 A3 is argued, not measured.
- Replication, allocator failover (§5.5), the mark-price pipeline (§2.5) and the waterfall below the unwind (§5.4) are designed and not built.
- E8 covers one two-factor scenario set; it supports “the mechanism holds for a richer finite set”, not the adequacy of any set. E9's breaker trips at half the widest scenario step, 10% on symbol A, matching Session 3's band; the halt duration is not modelled. The fall is larger than Session 3's 4% because a move inside the grid triggers no liquidation, the reopen

prices are assumptions, and the fund figure is one configuration, not a sizing rule.

- Superseded results — E4’s schedule comparison, E5’s suppression sweep — are in `experiments/superseded/` and are cited nowhere as current.

9.5 Section ownership

Each member owns the sections below: presents them at the defence and answers for them. The panel may ask any member any question, and all four are prepared to explain the admission condition of §2.2, including where the factor of two comes from.

§	Section	Owner
1	Business context and requirements	Zhou Congxiang
2	Architecture	Liu Kefan
3	Consistency map	Zhou Congxiang
4	Data and storage design	Zhang Haoxi
5.1–5.3	State, log, idempotency chain	Zhang Haoxi
5.4–5.7	Liquidation, waterfall, recovery	Wu Youjhen
6	Security and threat model	Wu Youjhen
7	Trade-offs and alternatives	Liu Kefan
8	Operations	Wu Youjhen
9, A, C, D	Disclosure and appendices	Liu Kefan
E, F	Liquidation detail, trust boundaries	Wu Youjhen

Appendix A — Decision history

This appendix holds what the main body no longer claims. It is here because a design document that quietly deletes its reversals is less trustworthy than one that records them, and because §9 has to disclose what was rejected as well as what was adopted.

Nothing in this appendix is current evidence. Where a figure appears, it was produced by an interface the mechanism has moved off, and the experiment that produced it now lives in `experiments/superseded/`.

A.1 What was withdrawn, and what replaced it

Withdrawn claim	Why it failed	What replaced it
A capacity schedule contracting with a published market state makes the mechanism safe	A lease cannot remove a position it has already admitted. Capacity that shrinks restricts the <i>next</i> admission, not the exposure already created	A flat ceiling for the term. The schedule survives only as a local operational trigger, with no capacity claim (ADR-2)
A state-contingent schedule recovers 3.8× the throughput of worst-case sizing	Measured on the incremental-charge mechanism, which did not bound the account’s requirement at all	Utilisation is reported directly: the closure caps it near half of equity, and E1’s binding trial reaches 99% of two envelopes at 49% of equity
Charging an order its marginal requirement is sufficient	A leg flipped from short to long leaves the local increment unchanged while the account’s requirement moves to its maximum (c1)	Admission compares absolute worst-fill envelopes
Gross notional is additive across the partition	Two gateways can hold opposite legs in the same symbol; they net inside the account	Gross is sub-additive by the triangle inequality, and the add-on is evaluated once on the summed gross (Appendix D.1)
Expiry releases the exposure a holder created	A term ends authority. The positions stay	Authority and committed exposure are tracked separately; only a terminal reconciliation or an account barrier lowers the second (§5.4)
A gateway may keep accepting risk-reducing orders during a partition	An order that lowers one gateway’s requirement can raise the account’s by removing a hedge held elsewhere (c9)	A gateway in quarantine admits nothing. Risk reduction is an account-level operation (§3.3)
A compromised gateway’s blast radius is bounded by its leases	Nothing downstream re-derives the envelopes the gateway compared against	The gateway is inside the trusted computing base, and §6.1 says what it can and cannot do
A lease id identifies authority	It was a bearer token: any account, any claimed holder	Registered bindings and session-resolved holders at the ordering point (<code>tests/test_authority.py</code>)

Withdrawn claim	Why it failed	What replaced it
The closure absorbs an equity overstatement of about 64%	Measured in a configuration where the ceiling was not binding	At the binding point the breach tracks the overstatement one for one (E5 Part B)
The lease term is the latency of every credit decision	Raising a limit takes effect at the next issuance	It is the enforcement latency of a <i>tightening</i> under partition (§1.5)

A.2 The abuse cases the schedule created

Retained verbatim because they are the record of what the market-data authority argument was, and because §6.1's weaker claim only makes sense against them. Under the current condition the admission check reads no market state, so A1 has no target on the admission path; A2 survives in the weakened form now recorded as §6.3 A1.

A1 — Replaying an older market state The shard evaluates its schedule at the market state it reads. An account able to make a shard read a stale, more favourable state obtains capacity the current state does not permit.

Mitigation: the shard evaluates its schedule at the most adverse state it has observed since the lease was installed, not at the state on the current message. A replayed lower state changes nothing.

Cost of the mitigation when the feed is honest: none. In E5 the ratcheted and unratcheted configurations are identical on an honest feed — 58 orders admitted, zero ticks with the requirement above equity. The defence only engages when the state goes backwards, which on an honest feed it does not.

A2 — Holding the published mark below the market An account able to suppress the published state, rather than replay an old one, is a different attack: the state never goes backwards, it simply fails to go forwards. The ratchet cannot see this, because there is nothing to ratchet against.

E5 measures it. The published state is held at 0 between ticks 20 and 50 while the checker uses the true state:

Configuration	Suppressed	Admitted	Ticks above equity, of 480
Scalar lease	no	36	236
Scalar lease	yes	36	236
Schedule, current state	no	58	0
Schedule, current state	yes	62	12
Schedule, ratcheted	no	58	0
Schedule, ratcheted	yes	49	5

Three readings. The scalar lease is indifferent to the attack because it never reads the state — it is also above equity 236 ticks out of 480 for unrelated reasons (§7). The schedule closes that, and in doing so opens A2: suppression buys 4 extra admissions and 12 ticks of exposure. The ratchet reduces the exposure to 5 ticks and costs 9 admissions under attack.

This is a partial mitigation and we do not present it as more than that. The residual belongs to the mark pipeline, not to the lease: multiple independent sources, a trimmed statistic across them, staleness detection on each source, and a bound on how far the published state may lag the sources. That pipeline is now designed in §2.5; this record is kept as the argument that led to it.

A.3 The counterexample ledger

Sixteen lifecycle counterexamples (`tests/test_counterexamples.py`, c1–c16), six worst-fill cases (w1–w6), six recovery cases (r1–r6), fourteen liquidation cases (l1–l14), six repricing cases (m1–m5 with m4 split), six authority cases (t1–t6), seven execution-cost cases (d1–d7) and fourteen ledger cases (a1–a14). Each was written after a failure, most of them found by external review, and each is named in the section of the main body that depends on it. The pattern worth extracting is that **every one of them was found by a check that compared against something recomputed independently**, never by a check that trusted a figure a component reported about itself.

The three that changed the mechanism rather than fixing an implementation are c1 (absolute envelopes), c9 (local risk reduction is not global) and m1 (gross measured at one mark). The rest are lifecycle discipline.

Appendix C — Protocols in full

The main body states what each protocol guarantees and cites the test that pins it. This appendix carries the step-by-step form, for a reader checking the guarantee against the mechanism rather than reading the argument.

C.1 The idempotency chain, in full

End to end, for one client order. Every link is a named identifier and a decision point that has been tested rather than argued.

1. **The client** assigns a client order ID and retries with the same ID on timeout. A timeout is an unknown outcome, not a failure.
2. **The gateway** checks its three envelopes and, if they hold, submits to the ordering point under `(lease_id, admission_seq)` where `admission_seq` is the next number it has used under that lease.
3. **The ordering point** accepts that pair only if it is the next number for the lease, and only if the lease is not fenced. A gateway resend carrying the same payload under the same pair succeeds again and records once; the same pair with a different payload is a conflict and records nothing. The gap-free sequence is what later lets a seal claim to cover every admission the lease produced.
4. **A fill** is submitted under a `fill_id` against an `order_id`. The ordering point refuses it — writing nothing and moving nothing — if the identifier has been used with different figures, the order is unknown or cancelled, the direction disagrees, the cumulative quantity would exceed what was admitted, the price falls outside the band recorded at admission, or the fee exceeds the cap for that quantity. A retry with the same figures lands once (`tests/test_execution_debit.py`, d4 to d6).
5. **The order of operations is fixed**: the ordering point decides first, and only a fill it accepted is folded into the gateway and the account. An earlier implementation called the gateway first, which moved state the authority then refused.
6. **A liquidation basket** is committed under a `basket_id` as a single record. A retry with the same payload is idempotent; the same identifier with different figures is refused and moves nothing (`tests/test_liquidation.py`, l7). A process that dies between the commit landing in the log and the transfer being folded in locally rebuilds from the log, and `applied_baskets` plus the ledger's fill keys make the reapplication land once (l14).
7. **Replay is idempotent by log position**. An earlier version relied on the order ID for that, which covers admissions and not fills or cancels, and replaying the same slice twice applied the fills twice (`tests/test_recovery.py`, r2).
8. **The matching shard** applies a given client order ID at most once, so no duplicate book action occurs regardless of how many times the order was admitted upstream.
9. **Clearing** derives ledger entry IDs deterministically from the trade sequence, so a replayed trade produces the same entry ID and is deduplicated (running case, Part 3 §4).

Step 2 is where this design differs from the running case, and it is a cost we state rather than hide: a client retry, at the same gateway or another, is a new admission attempt and may conservatively consume envelope there, though it cannot produce a duplicate book action. That consumption is released when the duplicate's rejection is recorded at the ordering point, not at the next issuance.

One asymmetry is deliberate and matters later. A cancel *request* releases nothing; only an acknowledgement recorded at the ordering point does, because until then the order can still fill. That produces two different failures which are not the same fact and are tested separately (Appendix E.2).

Exactly-once is, as in the running case, end-to-end idempotency layered on at-least-once delivery. Nothing in the margin path changes that.

C.2 The settlement barrier, in full

A seal is portable evidence for releasing **one** lease: the ordering point issues it when a lease is fenced, naming the last admission it recorded, and a holder carries it to the allocator. The account-wide compaction does not use one, because the allocator reads the same log the seal was cut from.

A seal releases one lease. A globally fenced, ordered account barrier permits account-wide compaction.

`Allocator.settle` requires all of the following, and none of them is supplied by the caller:

1. the account is in `settling`, so no new lease can be issued for it. Doing this first is what stops a lease minted after the barrier from being authority the settlement did not account for;
2. every lease ever minted for the account is fenced at the ordering point — every ingress lease, every incarnation, and the liquidator's own basket authority. `all_leases` is the set the barrier is taken over, not `authority`, because a lease that admitted nothing is still authority;
3. a barrier watermark `B` at which the recorded sequence under each of those leases is gap-free and no admission or basket for the account was recorded under a lease outside the set. `submit` and `commit_basket` already refuse anything but the next number, so this check should never fire; it is here because the settlement's whole claim is that the log is complete, and an assertion that never fires is cheap next to a claim that is assumed;
4. the aggregate rebuilt from the log at `B`: filled positions, orders with no authoritative cancel acknowledgement, worst-fill risk, repriced gross reach, and the execution cost still ahead of the account;
5. installation under a credit-version compare-and-set, so a settlement computed at an older barrier cannot overwrite a newer one;
6. issuance resumes only after the install.

E7 exercises the refusals as well as the successes: `no_fence` is refused with two ingress leases live, and `liquidator_authority_live` is refused with the liquidator's lease live even though every ingress lease is fenced and the account is flat.

C.3 Authority binding at the ordering point

The ordering point holds `lease_id -> (account, holder, kind)`, registered by the allocator at issuance because the allocator is the single issuer and therefore the only component that knows the binding. The holder on a submission is resolved from the authenticated session and never read from the request body. A submission is refused as `unknown_lease`, `wrong_account`, `wrong_holder`, `wrong_authority_kind` or `unauthenticated` before any sequence check runs.

`kind` is `ingress` or `liquidation` and they are not interchangeable: an ingress holder that could commit a basket would be admitting with no ceiling at all, since the liquidator's admissions are checked against the merged account instead.

Deliberately not checked here: the lease's term. The ordering point has no clock it can compare against an expiry set by another component, so an honest gateway is bounded by its own term and a Byzantine one only by the fence (§6.1, ADR-6).

`tests/test_authority.py` pins six cases, including the one an external reviewer demonstrated against the previous interface: a valid lease id submitted under a different account, which used to return `(True, "ok")`.

Appendix D — The mechanism, formally

§2.2–§2.3 state the mechanism in words. This appendix carries the definitions, the three lemmas, the worst-fill closed forms and the closure argument with its tightness, for a reader checking the claim rather than following the design. The tests named here are the evidence; the prose is the argument.

D.1 What can be divided, and what cannot

Write the account's requirement as a scenario term plus an add-on. Two gross figures are needed and they are not the same object:

$$R(P) = \max_{k \in S} \text{loss}_k(P), \quad G_k(P) = \sum_s |q_s| m_s(k), \quad G^+(P) = \sum_s |q_s| \max_{j \in S} m_s(j)$$

$$M_k(P) = R(P) + A(G_k(P)), \quad A \text{ convex, non-decreasing, } A(0) = 0$$

G_k is what the requirement is computed from; G^+ is what a lease reserves against, because the marks move during a term (D.2). $G_k(P) \leq G^+(P)$ for every k by construction. The code keeps them apart as `gross` and `gross_reach`.

R is the worst loss across a fixed scenario set S , and the loss under any single scenario is linear in positions. A is a concentration and liquidity add-on.

The algebra below holds for any finite S . Most correctness experiments here use seven points on a single factor with non-negative loadings; E8 repeats E1's check on seventeen scenarios with two factors, loadings of both signs and idiosyncratic moves, and finds no breach, and E3 additionally times a 16-point grid. Nothing here shows that any particular set is adequate for 40 underlyings; that is a calibration question, and its evidence is not in this document. What is shown is that the decomposition, the closure and the lifecycle are correct for whatever finite S is picked.

The partition is by gateway, not by symbol. Two gateways can hold opposite positions in the *same* symbol, and those net inside the account, so `gross` is not additive across the partition.

Lemma 1 — R is sub-additive. $R(P) = \max_k \sum_g \text{loss}_k(P_g) \leq \sum_g \max_k \text{loss}_k(P_g) = \sum_g R(P_g)$: a single scenario cannot beat the per-gateway worst cases taken separately.

Lemma 2 — gross is sub-additive. Per symbol, $|\sum_g q_{g,s}| \leq \sum_g |q_{g,s}|$ by the triangle inequality; multiplying by a positive mark and summing gives $G_k(\sum_g P_g) \leq \sum_g G_k(P_g)$ for every k , and the same for G^+ , with equality only when every gateway holds the same sign in every symbol.

Lemma 3 — A does not decompose. A convex function through the origin is super-additive on non-negative arguments, so $\sum_g A(G_g) \leq A(\sum_g G_g)$. Per-gateway add-on allowances added up under-state the whole, however the positions are split.

The decomposition rule follows rather than being chosen:

The sub-additive parts divide into per-gateway ceilings checked locally. The add-on does not; it is evaluated once, centrally, on the summed gross, and A being non-decreasing is what makes that an upper bound.

Chained, for the realised scenario k :

$$G_k(P') \leq G^+(P') \leq \sum_g G^+(P'_g) \leq \sum_g \lambda_g^G$$

the first step by construction, the second by Lemma 2, the third by the admission rule. A non-decreasing then carries it to $A(G_k(P')) \leq A(\sum_g \lambda_g^G)$. Nothing in that chain needs `gross` to be additive. Lemmas 1 and 3 are checked over 2,000 sampled portfolios in `tests/test_algebra.py`, in the predicted directions.

D.2 The envelopes and the closure

A lease cannot undo an admission it has already granted. Everything here follows from that.

Three envelopes

Envelope	What it bounds	Why separate
λ_g^R	worst-fill scenario requirement of everything the gateway holds	sub-additive, so it divides
λ_g^G	worst-fill G^+ the gateway can reach	an order can lower R while raising gross
λ_g^D	execution cost the gateway can still incur, plus cost already incurred that the equity the current lease was solved against does not yet reflect	its effect on equity is covered by neither the scenario requirement nor the gross add-on

The three are not three allocations: λ^G and λ^D are issued at fixed ratios to λ^R , so the solver searches one scalar along a ray through a three-dimensional feasible set. Dropping the second half of λ^D is what made capacity decay every term in an earlier implementation (d7).

What a gateway holds is orders, not positions Two resting orders of opposite sign net to nothing, and if only one fills the account carries the other side. The envelopes are taken over the worst subset of fills that could still occur, which needs no enumeration because the loss under a fixed scenario is linear:

$$E_k = \text{loss}_k(f) + \sum_i \max(0, \text{loss}_k(o_i)), \quad R_{\text{wf}} = \max\left(0, \left\lceil \frac{1}{D} \max_k E_k \right\rceil\right)$$

$$G_{\text{wf}} = \sum_s \left(\max_j m_s(j) \right) \max(|f_s + b_s|, |f_s - s_s|)$$

with f the filled position, o_i the live orders, and b_s, s_s the unfilled quantity on each side.

Both closed forms agree with enumeration of all 2^n fill subsets on 4,000 random books (`test_worst_fill_exhaustive`). Admission compares these **absolute** figures against the ceilings, not the increment an order adds: a leg flipped from short to long leaves the increment unchanged while the account's requirement moves to its maximum (c1).

Where gross is measured A lease reserves against G^+ , not against the gross standing when the solve ran. A short position's adverse scenario raises the mark, raises gross and raises the add-on, while a figure measured at the issuance mark does not move. Reserving at the issuance mark admits 296 lots in the worked case of m1 and finishes 382,143 above equity; reserving at G^+ admits 249 and finishes 5,842 inside. ADR-3 gives the cost and the scope of the tightness claim.

The condition and its closure

$$2 \sum_g \lambda_g^R + A \left(\sum_g \lambda_g^G \right) + \sum_g \lambda_g^D \leq E_0$$

with $E_0 = \text{Collateral} + \sum_s (c_s + q_s m_s) - \text{fees}$, mark-to-market equity rather than collateral. There is no market state in it.

Write P' for the position the term ends with, D for the execution cost it incurred, k for the realised scenario. Three bounds come from the admission rule and the fourth because R is a maximum over a set containing k :

$$R(P') \leq \sum_g \lambda_g^R, \quad G_k(P') \leq \sum_g \lambda_g^G, \quad D \leq \sum_g \lambda_g^D, \quad \text{loss}_k(P') \leq R(P')$$

Adding requirement, cost and realised loss:

$$M_k(P') + D + \text{loss}_k(P') \leq 2 \sum_g \lambda_g^R + A \left(\sum_g \lambda_g^G \right) + \sum_g \lambda_g^D \leq E_0$$

and since $\text{Equity_after}(k) = E_0 - D - \text{loss}_k(P')$,

$$M_k(P') \leq E_0 - D - \text{loss}_k(P') = \text{Equity_after}(k)$$

Subtracting D is why the third resource exists. A conclusion of $M \leq E_0 - \text{loss}$ would leave execution cost out of the arithmetic while still listing λ^D as a ceiling.

The factor of two is a closure, not a margin, and it is tight. Set $A = D = 0$ and take $R(P') = \lambda^R$ with the realised scenario attaining the maximum, so $\text{loss}_k = R$. Then $M_k = \lambda^R$ and $\text{Equity}_{\text{after}} = E_0 - \lambda^R$, equal precisely when $E_0 = 2\lambda^R$. With $c < 2$ the solve issues $\lambda^R = E_0/c$, and the same position ends with $M_k = E_0/c > E_0 - E_0/c$. The coefficient cannot be reduced.

What the closure costs Utilisation is capped near half of equity before the add-on reserve. In E1's binding trial — every order filled at the worst price and fee the policy allows — the risk and debit envelopes reach 99% and the requirement is 49% of equity, with no breach. The offset decomposition gives up is separately D.1's sub-additivity gap (§7). The multi-factor case is E8.

Appendix E — Liquidation and settlement flow

The step-by-step flow behind §5.4, moved here from §2.9 to keep the body to its page budget. E7 injects a fault at each of its three stages.

E.1 Figure 5 — Liquidation and settlement

The path from a shortfall to capacity being returned, in three stages. E7 injects a fault at each.

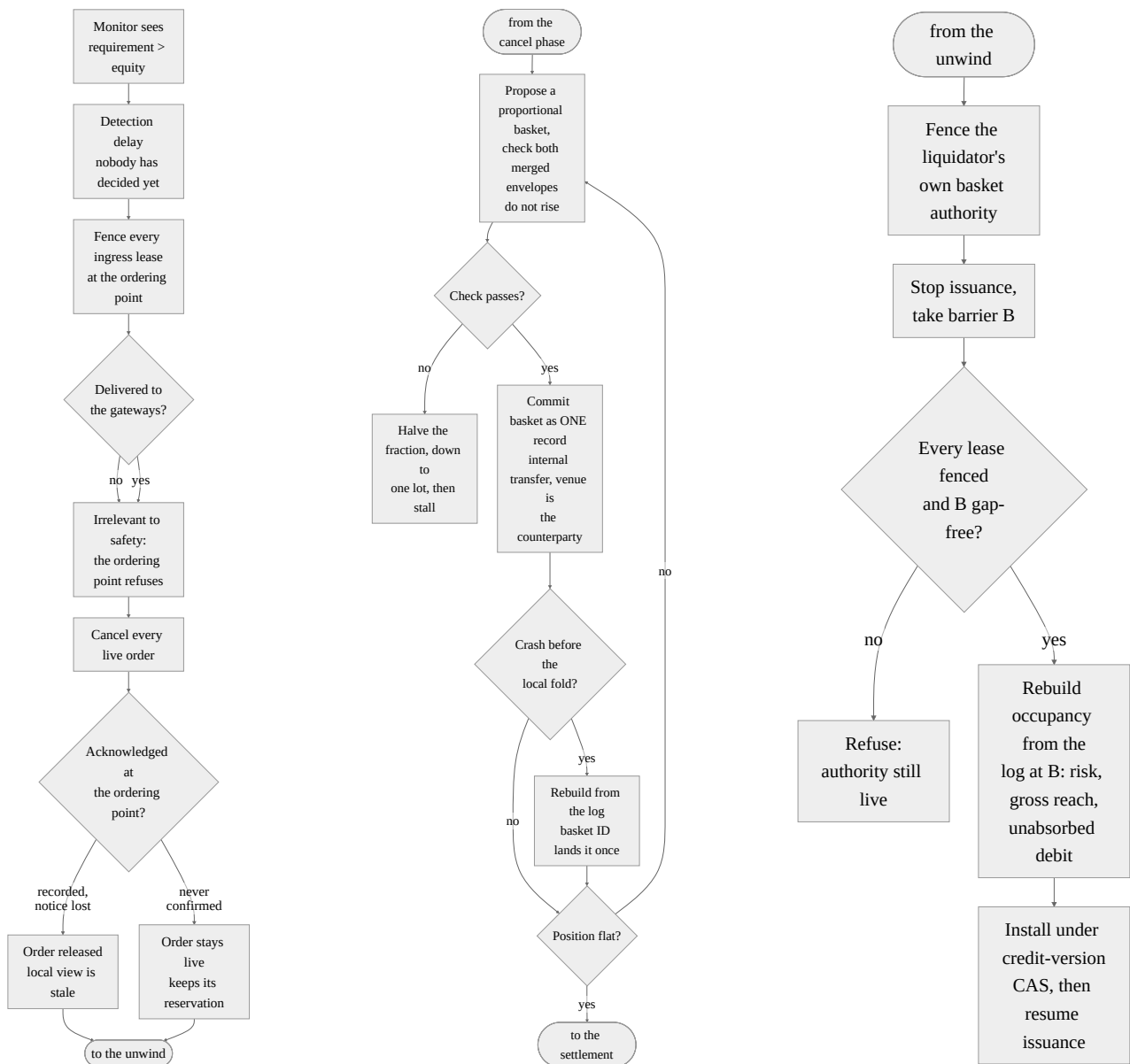


Figure 5: Liquidation and settlement. Stop, unwind, settle. The two exits from the cancel check are the two different facts of §5.4.

The cancel check has two labelled exits because they are two different facts (E.2): a cancel recorded at the ordering point releases the order, one the matching side never confirmed does not. The barrier refuses on `no_fence` and on a live liquidator, and E7 exercises both refusals.

E.2 What a cancel and a delay cost

Two ways a cancel fails

Failure	What the log holds	What the settlement must do
Acknowledgement recorded, notification to the gateway lost	the cancel	release the order; only the local view is stale
Matching side never confirmed	nothing	keep the worst-fill reservation and the execution-cost reserve

Fencing does not help in the second case: it stops new admissions and does nothing to a resting order. Both arms of E7 show one order live at the end; the settlement figure is $(0, 0, 0)$ for the recorded cancel and $(120, 2232, 9)$ for the unacknowledged one.

What the delay costs E6 splits the equity change exactly — ending equity = trigger equity + drift – slippage – fees, asserted on integers with no tolerance. Across its delay sweep execution cost runs $2,522 \rightarrow 6,486$ while market drift runs $16,444 \rightarrow 229,708$. **What the mechanism controls — new admissions and the cost of unwinding — is bounded; market drift is not.** Without a fence the unwind’s cost exceeds its bound by 1,562. E6’s figures are one configuration, one seed, one price path.

Appendix F — Trust boundaries in full

The argument behind §6.1, moved here to keep the body to its page budget.

The gateway is inside the trusted computing base. An earlier version of this document claimed its blast radius is bounded by the leases it holds. That is withdrawn. Nothing downstream re-derives the envelopes: the ordering point checks that an admission is the next number for a live, correctly bound lease and that a fill matches the terms recorded at admission; it does not recompute the worst-fill figures the gateway compared against its ceilings. **A gateway that lies about its own envelope arithmetic is believed.**

What a compromised gateway cannot do is each pinned by a test. It cannot act under a fenced lease or under a lease id the allocator never minted (t3, t4). It cannot use another account’s lease or another holder’s, or submit without an authenticated session (t1, t2). It cannot commit a liquidation basket under an ingress lease (t5). It cannot fill outside the band or above the fee cap recorded at admission, fill in the wrong direction, over-fill, or land the same fill twice (d4 to d6). Nor can it hide an admission, because the ordering point takes only the next sequence number for its lease.

What it can do, stated without softening: **a compromised gateway is bounded by neither its ceilings nor its own term.** It can submit arbitrary quantity for every account whose lease it holds, and it will not stop at its expiry, because the ordering point does not enforce terms — it has no clock it can compare against an expiry set elsewhere (ADR-6). A term bounds an *honest* gateway that has lost contact. The only thing that stops a dishonest one is a fence committing at the ordering point.

So the mechanism bounds the **scope** of the damage — which accounts, which authority kind, which holder — and does not bound its **magnitude**. The exposure window is detection latency plus fence-commit latency, and nothing in this document measures either. Closing this would mean re-deriving the envelopes at the ordering point, which puts the per-order margin computation back on the single-writer path — the thing §2.2 exists to avoid. That trade is not made here; it is the residual.

The liquidator is inside it too, on its own account. Its orders are checked against the merged account rather than a ceiling (c9), so no ceiling bounds it. The non-increase test bounds the *risk* it creates — neither merged envelope may rise — but permits unlimited churn, and churn costs execution. Its authority ends at the ordering point like any other’s, and the barrier refuses to run while it is live (t5, l11): containment of duration, not of authority.

The market-data path carries no authority in the running case. Here marks set equity, the scenario displacements and G^+ , so a wrong value changes how much capacity is solved for — weaker than an earlier draft claimed, since the admission check reads no market state, but it is the exposure in §6.3 A1.